

图论强化训练

全源最短路径与 Johnson 算法专项练习 (C++)

涵盖内容：

- ✓ 动态规划视角：Floyd-Warshall
- ✓ 邻接矩阵、后继矩阵与路径恢复
- ✓ 负权环检测： $d[i][i] < 0$
- ✓ 基线方案：重复调用 Dijkstra
- ✓ 势函数与图的重赋权 $w' = w + h(u) - h(v)$
- ✓ Johnson：Bellman-Ford + $|V|$ 次 Dijkstra
- ✓ 稀疏图与稠密图的复杂度取舍

语言：C++ 请先完成引导性问题，再补全程序并运行测试

预备知识：全源最短路径问题

设带权有向图 $G = (V, E)$ ，边 (u, v) 的权值为 $w(u, v)$ ，记 $n = |V|$ ， $m = |E|$ 。全源最短路径问题（All-Pairs Shortest Paths, APSP）要求计算矩阵

$$D = (\delta(u, v))_{n \times n}, \quad \delta(u, v) = \text{从 } u \text{ 到 } v \text{ 的最短路径权值}.$$

与单源问题不同，APSP 的输出规模本身就是 $\Theta(n^2)$ ，因此任何算法都不可能快于 $\Omega(n^2)$ 。

四种常见方案的比较如下：

方案	负权边	时间复杂度	适合的图
重复 Bellman-Ford	允许	$O(n^2m)$	一般不用，作为对照
Floyd-Warshall	允许	$O(n^3)$	稠密图， $m = \Theta(n^2)$
重复 Dijkstra	不允许	$O(nm \log n)$	稀疏非负权图
Johnson	允许	$O(nm \log n + nm)$	稀疏图且含负权边

当 $m = \Theta(n^2)$ 时， $O(nm \log n) = O(n^3 \log n)$ 反而不如 Floyd-Warshall；当 $m = \Theta(n)$ 时，Johnson 的 $O(n^2 \log n)$ 显著优于 $O(n^3)$ 。稀疏还是稠密，是选择 APSP 算法的首要依据。

Johnson 算法的核心是**重赋权**：找到一组顶点势 $h: V \rightarrow \mathbb{R}$ ，令

$$w'(u, v) = w(u, v) + h(u) - h(v), \quad (1)$$

使所有 $w'(u, v) \geq 0$ ，从而可以在新图上使用 Dijkstra。对任意从 u_0 到 u_k 的路径 $p = \langle u_0, u_1, \dots, u_k \rangle$ ，中间项两两相消（望远镜求和）：

$$w'(p) = \sum_{i=1}^k (w(u_{i-1}, u_i) + h(u_{i-1}) - h(u_i)) = w(p) + h(u_0) - h(u_k). \quad (2)$$

由于偏移量 $h(u_0) - h(u_k)$ 只与端点有关，与具体路径无关，因此**重赋权**不改变任何一对顶点之间的最短路径集合，只需在最后把距离还原：

$$\delta(u, v) = \delta'(u, v) - h(u) + h(v). \quad (3)$$

第 1 题 稠密图全源最短路径（Floyd-Warshall）

考点：动态规划 · 中间顶点集合 · 三重循环顺序 · 后继矩阵 · 负权环检测 · 溢出处理

1.1 背景知识

把顶点编号为 $0, 1, \dots, n-1$ 。定义

$d^{(k)}[i][j]$ = 只允许使用 $\{0, 1, \dots, k-1\}$ 作为中间顶点时, $i \rightarrow j$ 的最短路径权值。

考虑最优解是否使用顶点 k 作为中间点, 得到递推式

$$\begin{cases} d^{(0)}[i][j] = \begin{cases} 0, & i = j, \\ w(i, j), & (i, j) \in E, \\ +\infty, & \text{其他}, \end{cases} \\ d^{(k+1)}[i][j] = \min\{d^{(k)}[i][j], d^{(k)}[i][k] + d^{(k)}[k][j]\}. \end{cases} \quad (4)$$

最终答案为 $D = d^{(n)}$ 。由于 $d^{(k+1)}[i][k] = d^{(k)}[i][k]$ 且 $d^{(k+1)}[k][j] = d^{(k)}[k][j]$ (第 k 行、第 k 列在第 k 轮不会变化), 可以原地使用同一个二维数组, 无需滚动数组。

路径恢复可以维护后继矩阵 $\text{nxt}[i][j]$, 表示 $i \rightarrow j$ 的最短路径上 i 的下一个顶点。松弛成功时令 $\text{nxt}[i][j] \leftarrow \text{nxt}[i][k]$ 。

算法结束后, 若存在 $d[i][i] < 0$, 说明顶点 i 落在一个负权环上。

时间复杂度 $O(n^3)$, 空间复杂度 $O(n^2)$ 。

1.2 引导性问题

1. 为什么三重循环中 k 必须放在最外层? 若写成 `for i / for j / for k` 的顺序, 递推式 (4) 的哪个前提被破坏了?
2. 递推式 (4) 中的 $d^{(k)}$ 被原地覆盖, 为什么不会读到“未来”的值? 请结合第 k 行与第 k 列在第 k 轮保持不变来说明。
3. 用整型常量 `INF` 表示不可达时, `dist[i][k] + dist[k][j]` 可能发生什么问题? 为什么必须先判断两者都不等于 `INF`? 若把 `INF` 取为 `LLONG_MAX` 会有什么后果?
4. 存在负权边时, 为什么 $d[i][i]$ 可能变成负数? $d[i][i] < 0$ 与“图中存在负权环”是等价的吗? i 一定在环上吗?
5. 图中存在负权环时, 按 `nxt` 恢复路径可能出现什么现象? 程序应当如何防御?
6. 用邻接矩阵存图时, 若输入包含重边 $(u, v, 3)$ 与 $(u, v, 5)$, 应当保留哪一条? 若包含自环 $(u, u, -2)$, 应当如何处理?
7. Floyd-Warshall 的循环体只有一次比较和一次加法。相比重复 Dijkstra 的堆操作, 这带来了什么常数因子上的优势?

1.3 题目要求

实现以下成员函数：

1. `addEdge(u, v, w)`：添加有向边，重边保留权值较小者。
2. `run()`：执行 Floyd-Warshall，同时维护 `nxt_` 矩阵。
3. `distance(i, j)`：返回最短距离，不可达返回 `INF`。
4. `path(i, j)`：返回顶点序列，不可达返回空向量。
5. `hasNegativeCycle()`：判断是否存在负权环。

1.4 接口参考（仅声明，补全实现）

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 constexpr long long INF = 1LL << 60; // 足够大，且加法不溢出
6
7 class FloydWarshall {
8 public:
9     explicit FloydWarshall(int n)
10         : n_(n),
11           dist_(n, std::vector<long long>(n, INF)),
12           nxt_(n, std::vector<int>(n, -1)) {
13         // TODO: 初始化对角线为 0
14     }
15
16     // 添加有向边 u -> v, 重边取权值最小者
17     void addEdge(int u, int v, long long w) {
18         // TODO
19     }
20
21     // 执行 Floyd-Warshall, 同时填写后继矩阵
22     // 提示:
23     // 1. k 在最外层, i、j 在内层;
24     // 2. 松弛前判断 dist_[i][k] != INF && dist_[k][j] != INF;
25     // 3. 松弛成功时 nxt_[i][j] = nxt_[i][k]。
26     void run() {
27         // TODO

```

```

28     }
29
30     long long distance(int i, int j) const {
31         // TODO
32         return INF;
33     }
34
35     // 恢复 i -> j 的最短路径；不可达返回 {}
36     // 提示：从 i 出发反复跳 nxt_，直到到达 j；
37     //       为防止负权环导致死循环，可限制步数不超过 n_。
38     std::vector<int> path(int i, int j) const {
39         // TODO
40         return {};
41     }
42
43     // 是否存在负权环：检查是否有 dist_[i][i] < 0
44     bool hasNegativeCycle() const {
45         // TODO
46         return false;
47     }
48
49 private:
50     int n_;
51     std::vector<std::vector<long long>> dist_;
52     std::vector<std::vector<int>> nxt_;
53 };

```

1.5 测试用例

```

1 void printMatrix(const FloydWarshall& g, int n) {
2     for (int i = 0; i < n; ++i) {
3         for (int j = 0; j < n; ++j) {
4             long long d = g.distance(i, j);
5             if (d >= INF) std::cout << " inf";
6             else std::cout << ' ' << d;
7         }
8         std::cout << '\n';
9     }
10 }

```

```
11
12 int main() {
13     // 情形 1: CLRS 经典含负权边有向图, 无负权环
14     // 0->1:3 0->2:8 0->4:-4 1->3:1 1->4:7
15     // 2->1:4 3->0:2 3->2:-5 4->3:6
16     FloydWarshall g(5);
17     g.addEdge(0, 1, 3);
18     g.addEdge(0, 2, 8);
19     g.addEdge(0, 4, -4);
20     g.addEdge(1, 3, 1);
21     g.addEdge(1, 4, 7);
22     g.addEdge(2, 1, 4);
23     g.addEdge(3, 0, 2);
24     g.addEdge(3, 2, -5);
25     g.addEdge(4, 3, 6);
26     g.run();
27
28     printMatrix(g, 5);
29     // 期望距离矩阵:
30     // 0  1  -3  2  -4
31     // 3  0  -4  1  -1
32     // 7  4  0  5  3
33     // 2  -1 -5  0  -2
34     // 8  5  1  6  0
35
36     std::cout << g.hasNegativeCycle() << '\n'; // 0
37     std::cout << g.distance(0, 2) << '\n'; // -3
38     for (int v : g.path(0, 2)) std::cout << v << ' ';
39     std::cout << '\n'; // 0 4 3 2
40
41     // 情形 2: 存在负权环 (1 -> 2 -> 1, 权值和 -2)
42     FloydWarshall neg(3);
43     neg.addEdge(0, 1, 1);
44     neg.addEdge(1, 2, -1);
45     neg.addEdge(2, 1, -1);
46     neg.run();
47     std::cout << neg.hasNegativeCycle() << '\n'; // 1
48     std::cout << (neg.distance(1, 1) < 0) << '\n'; // 1
49
50     // 情形 3: 部分顶点不可达
```

```

51 FloydWarshall dis(4);
52 dis.addEdge(0, 1, 5);
53 dis.addEdge(2, 3, 2);    // 与 {0,1} 不连通
54 dis.run();
55 std::cout << dis.distance(0, 1) << '\n';    // 5
56 std::cout << (dis.distance(0, 2) >= INF) << '\n'; // 1
57 std::cout << dis.path(0, 3).size() << '\n'; // 0
58
59 // 情形 4: 单顶点
60 FloydWarshall one(1);
61 one.run();
62 std::cout << one.distance(0, 0) << '\n'; // 0
63 for (int v : one.path(0, 0)) std::cout << v << ' ';
64 std::cout << '\n';    // 0
65
66 // 情形 5: 重边取最小
67 FloydWarshall multi(2);
68 multi.addEdge(0, 1, 7);
69 multi.addEdge(0, 1, 3);
70 multi.run();
71 std::cout << multi.distance(0, 1) << '\n'; // 3
72 return 0;
73 }

```

第 2 题 基线方案：重复调用 Dijkstra

考点：以每个顶点为源点 • 最小堆与 `greater<>` • 惰性删除 • 复杂度对比

2.1 背景知识

若图中所有边权非负，最直接的 APSP 做法就是以每个顶点为源点各跑一次 Dijkstra：

$$D[s][\cdot] = \text{DIJKSTRA}(G, s), \quad s = 0, 1, \dots, n-1. \quad (5)$$

使用二叉堆时单次 Dijkstra 为 $O((n+m)\log n)$ ，总复杂度 $O(n(n+m)\log n)$ ，稀疏图上写作 $O(nm\log n)$ 。

在 C++ 中，`std::priority_queue` 默认是最大堆，需要显式指定比较器才能得到最小堆：

```

1 using State = std::pair<long long, int>; // (dist, vertex)
2 std::priority_queue<State, std::vector<State>,

```

3

```
std::greater<State>> pq;
```

由于标准库堆不支持”减键”，同一顶点可能多次入堆，出堆时需要惰性删除：若弹出的距离大于当前 `dist[u]`，说明该条目已过期，直接跳过。

本题的结果将作为第 4 题 Johnson 算法的正确性对照。

2.2 引导性问题

1. 为什么 `std::pair<long long, int>` 作为堆元素能得到正确的比较顺序？若把二元组写成 `(vertex, dist)` 会怎样？
2. 惰性删除下堆中最多有多少个元素？为什么复杂度仍是 $O((n+m) \log n)$ 而不是 $O(n^2 \log n)$ ？
3. 稠密图 $m = \Theta(n^2)$ 时，重复 Dijkstra 与 Floyd-Warshall 哪个更快？请代入复杂度公式比较，并考虑常数因子与缓存局部性。
4. 若图中出现一条负权边，重复 Dijkstra 的结果会错在哪里？请在测试用例的图上构造一个具体反例。
5. 若只需要少数几个源点（例如 $k \ll n$ 个）到所有顶点的距离，应当选择哪种方案？此时 Floyd-Warshall 浪费了什么？
6. $n = 10^5$ 、 $m = 3 \times 10^5$ 时，完整输出 $n \times n$ 的距离矩阵需要多少内存？这说明 APSP 在大图上有什么本质限制？

2.3 题目要求

实现以下成员函数：

1. `addEdge(u, v, w)`：添加边；若 $w < 0$ ，抛出 `std::invalid_argument`；`undirected_` 为真时同时添加反向边。
2. `dijkstra(source, parent)`：最小堆 + 惰性删除，返回距离数组并填写前驱数组。
3. `allPairs()`：返回 $n \times n$ 距离矩阵。
4. `path(s, t)`：调用一次 `dijkstra` 后恢复路径。

2.4 接口参考（仅声明，补全实现）

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <stdexcept>
5 #include <utility>
```



```
6
7 constexpr long long INF = 1LL << 60;
8
9 class RepeatedDijkstra {
10 public:
11     RepeatedDijkstra(int n, bool undirected = false)
12         : n_(n), undirected_(undirected), adj_(n) {}
13
14     // 添加边 u -> v, 要求 w >= 0, 否则抛 std::invalid_argument
15     void addEdge(int u, int v, long long w) {
16         // TODO
17     }
18
19     // 单源 Dijkstra: 最小堆 + 惰性删除
20     // 返回 dist, 并把前驱写入 parent (不可达为 -1)
21     std::vector<long long> dijkstra(int source,
22                                     std::vector<int>& parent) const {
23         // TODO
24         return {};
25     }
26
27     // 对每个源点各跑一次 Dijkstra, 拼成 n x n 距离矩阵
28     std::vector<std::vector<long long>> allPairs() const {
29         // TODO
30         return {};
31     }
32
33     // 恢复 s -> t 的最短路径; 不可达返回 {}
34     std::vector<int> path(int s, int t) const {
35         // TODO
36         return {};
37     }
38
39 private:
40     int n_;
41     bool undirected_;
42     std::vector<std::vector<std::pair<int, long long>>> adj_;
43 };
```

2.5 测试用例

```

1 int main() {
2     // 情形 1: 有向环状图, 验证绕行更短
3     // 0->1:1 1->2:2 2->3:3 3->0:4 0->2:10
4     RepeatedDijkstra g(4);
5     g.addEdge(0, 1, 1);
6     g.addEdge(1, 2, 2);
7     g.addEdge(2, 3, 3);
8     g.addEdge(3, 0, 4);
9     g.addEdge(0, 2, 10);
10
11     auto d = g.allPairs();
12     // 期望:
13     // 0 1 3 6
14     // 9 0 2 5
15     // 7 8 0 3
16     // 4 5 7 0
17     for (auto& row : d) {
18         for (long long x : row) std::cout << x << ' ';
19         std::cout << '\n';
20     }
21     for (int v : g.path(0, 3)) std::cout << v << ' ';
22     std::cout << '\n';          // 0 1 2 3 (权值 6, 优于 0->2->3 = 13)
23
24     // 情形 2: 无向三角形, 距离矩阵应对称
25     RepeatedDijkstra u(3, true);
26     u.addEdge(0, 1, 1);
27     u.addEdge(1, 2, 2);
28     u.addEdge(0, 2, 4);
29     auto du = u.allPairs();
30     // 0 1 3
31     // 1 0 2
32     // 3 2 0
33     std::cout << du[0][2] << ' ' << du[2][0] << '\n'; // 3 3
34
35     // 情形 3: 不可达
36     RepeatedDijkstra sp(4);
37     sp.addEdge(0, 1, 5);
38     sp.addEdge(2, 3, 2);

```

```

39  auto ds = sp.allPairs();
40  std::cout << (ds[0][2] >= INF) << '\n'; // 1
41  std::cout << ds[2][3] << '\n'; // 2
42
43  // 情形 4: 单顶点
44  RepeatedDijkstra one(1);
45  std::cout << one.allPairs()[0][0] << '\n'; // 0
46
47  // 情形 5: 负权边应被拒绝
48  try {
49      g.addEdge(1, 0, -1);
50  } catch (const std::invalid_argument& e) {
51      std::cout << e.what() << '\n'; // Dijkstra 不允许负权边
52  }
53  return 0;
54  }

```

第 3 题 Johnson 的第一步：势函数与图的重赋权

考点：虚拟源点 • Bellman-Ford 求势 • 三角不等式 • 非负性证明 • 望远镜求和

3.1 背景知识

Johnson 算法的关键是构造势函数 h 。做法是新增一个虚拟源点 q ，从 q 向每个顶点连一条权值为 0 的边，得到扩展图 G' ：

$$V' = V \cup \{q\}, \quad E' = E \cup \{(q, v, 0) \mid v \in V\}. \quad (6)$$

在 G' 上以 q 为源点运行 Bellman-Ford，令

$$h(v) = \delta_{G'}(q, v). \quad (7)$$

由于 q 能到达所有顶点，每个 $h(v)$ 都是有限值（前提是无负权环）。最短距离满足三角不等式

$$h(v) \leq h(u) + w(u, v), \quad \forall (u, v) \in E, \quad (8)$$

移项即得重赋权的非负性：

$$w'(u, v) = w(u, v) + h(u) - h(v) \geq 0. \quad (9)$$

若 G 中存在负权环, 则该环在 G' 中从 q 可达, Bellman-Ford 会检测到它, 此时 APSP 无解。

实现上不必真的建出第 $n+1$ 个顶点: 把所有 h 初始化为 0 再做 n 轮全边松弛, 效果与从虚拟源点出发完全相同。请在引导性问题中说明为什么。

3.2 引导性问题

1. 为什么不能任选一个真实顶点作为 Bellman-Ford 的源点? 若某顶点从该源点不可达, h 会取到什么值, 式 (9) 还成立吗?
2. 请从三角不等式 (8) 出发, 完整写出 $w'(u, v) \geq 0$ 的推导。
3. 把所有 $h[v]$ 初始化为 0、再做 n 轮松弛, 为什么等价于从虚拟源点出发? 虚拟源点带来的 n 条 0 权边在初始化中体现在哪里?
4. 加入虚拟源点后, 图中的最短路径集合、可达性和负权环情况分别有没有被改变? 为什么 q 没有入边这一点很重要?
5. 若原图所有边权非负, h 会等于什么? 此时重赋权后的权值与原权值有何关系?
6. 由式 (2) 可知 $w'(p) = w(p) + h(u_0) - h(u_k)$ 。请解释: 为什么这保证了“在 G' 中最短的路径在 G 中也最短”? 若偏移量依赖路径本身而非端点, 结论还成立吗?
7. 重赋权会不会把负权边变成正权、把正权边变成 0? $w'(u, v) = 0$ 说明边 (u, v) 具有什么性质?

3.3 题目要求

实现以下成员函数:

1. `addEdge(u, v, w)`: 把边加入边表。
2. `computePotentials()`: 等价于虚拟源点上的 Bellman-Ford, 返回势向量 h ; 若存在负权环, 抛出 `std::runtime_error("图中存在负权环")`。
3. `reweight(h)`: 返回重赋权后的边表。
4. `verifyNonNegative(h)`: 检查所有 $w'(u, v) \geq 0$ 。

3.4 接口参考 (仅声明, 补全实现)

```
1 #include <iostream>
2 #include <vector>
3 #include <stdexcept>
4
```

```
5 struct Edge {
6     int u, v;
7     long long w;
8 };
9
10 class Reweighting {
11 public:
12     explicit Reweighting(int n) : n_(n) {}
13
14     void addEdge(int u, int v, long long w) {
15         // TODO
16     }
17
18     // 返回势函数 h，等价于在虚拟源点上跑 Bellman-Ford
19     // 提示：
20     // 1. h 全部初始化为 0（相当于 q 到每点的 0 权边）；
21     // 2. 做 n_ - 1 轮全边松弛，用 updated 标记提前终止；
22     // 3. 再做一轮，若仍能松弛则抛 std::runtime_error("图中存在负权环")。
23     std::vector<long long> computePotentials() const {
24         // TODO
25         return {};
26     }
27
28     // 按  $w'(u,v) = w(u,v) + h[u] - h[v]$  生成新边表
29     std::vector<Edge> reweight(const std::vector<long long>& h) const {
30         // TODO
31         return {};
32     }
33
34     // 校验重赋权后所有边权非负
35     bool verifyNonNegative(const std::vector<long long>& h) const {
36         // TODO
37         return false;
38     }
39
40 private:
41     int n_;
42     std::vector<Edge> edges_;
43 };
```

3.5 测试用例

```

1 int main() {
2     // 情形 1: CLRS 经典图, 与第 1 题同一张图
3     Reweighting r(5);
4     r.addEdge(0, 1, 3);
5     r.addEdge(0, 2, 8);
6     r.addEdge(0, 4, -4);
7     r.addEdge(1, 3, 1);
8     r.addEdge(1, 4, 7);
9     r.addEdge(2, 1, 4);
10    r.addEdge(3, 0, 2);
11    r.addEdge(3, 2, -5);
12    r.addEdge(4, 3, 6);
13
14    auto h = r.computePotentials();
15    for (long long x : h) std::cout << x << ' ';
16    std::cout << '\n';          // 0 -1 -5 0 -4
17
18    std::cout << r.verifyNonNegative(h) << '\n'; // 1
19
20    for (const Edge& e : r.reweight(h))
21        std::cout << e.u << "->" << e.v << ':' << e.w << '\n';
22    // 0->1:4 0->2:13 0->4:0
23    // 1->3:0 1->4:10 2->1:0
24    // 3->0:2 3->2:0 4->3:2
25
26    // 情形 2: 全部非负权, 势函数应为全零
27    Reweighting pos(3);
28    pos.addEdge(0, 1, 2);
29    pos.addEdge(1, 2, 3);
30    auto hp = pos.computePotentials();
31    for (long long x : hp) std::cout << x << ' ';
32    std::cout << '\n';          // 0 0 0
33
34    // 情形 3: 负权环必须被检测到 (即使它与其他顶点不连通)
35    Reweighting bad(4);
36    bad.addEdge(0, 1, 1);
37    bad.addEdge(2, 3, -2);
38    bad.addEdge(3, 2, -2); // 2 <-> 3 构成负权环

```

```

39     try {
40         bad.computePotentials();
41     } catch (const std::runtime_error& e) {
42         std::cout << e.what() << '\n'; // 图中存在负权环
43     }
44
45     // 情形 4: 非连通图, 虚拟源点保证所有 h 都是有限值
46     Reweighting split(4);
47     split.addEdge(0, 1, -3);
48     split.addEdge(2, 3, -5);
49     auto hs = split.computePotentials();
50     for (long long x : hs) std::cout << x << ' ';
51     std::cout << '\n'; // 0 -3 0 -5
52     std::cout << split.verifyNonNegative(hs) << '\n'; // 1
53     return 0;
54 }

```

注意情形 3: 负权环即使从任何真实顶点都不可达, 也会让 APSP 中某些点对无解, 因此 Johnson 必须报告任意负权环。这与单源 Bellman-Ford “只报告源点可达的负权环” 不同, 请在引导性问题 1 的答案中一并说明原因。

第 4 题 Johnson 算法完整实现

考点: Bellman-Ford + n 次 Dijkstra • 距离还原 • 不可达处理 • 稀疏图复杂度

4.1 背景知识

Johnson 算法把 “允许负权” 的困难交给一次 Bellman-Ford, 把 “ n 次单源计算” 的负担交给快速的 Dijkstra:

$$\left\{ \begin{array}{l}
 1. \text{ 虚拟源点 Bellman-Ford 求 } h, \text{ 若有负权环则报错; } O(nm) \\
 2. \text{ 重赋权 } w'(u, v) = w(u, v) + h(u) - h(v) \geq 0; O(m) \\
 3. \text{ 对每个 } u \text{ 在 } w' \text{ 上跑 Dijkstra 得 } \delta'(u, \cdot); O(nm \log n) \\
 4. \text{ 还原 } \delta(u, v) = \delta'(u, v) - h(u) + h(v). O(n^2)
 \end{array} \right. \quad (10)$$

总复杂度 $O(nm \log n)$ 。当 $m = O(n)$ 时为 $O(n^2 \log n)$, 优于 Floyd-Warshall 的 $O(n^3)$; 当 $m = \Theta(n^2)$ 时退化为 $O(n^3 \log n)$, 此时应改用 Floyd-Warshall。

第 4 步有一个容易被忽略的细节: 若 $\delta'(u, v) = +\infty$, 则 $\delta(u, v) = +\infty$, 不能把 INF 代入减法运算, 否则会得到一个看起来有限的错误数值。

4.2 引导性问题

1. 请写出 $\delta(u, v) = \delta'(u, v) - h(u) + h(v)$ 的推导, 说明它是式 (2) 的直接推论。
2. 为什么 Dijkstra 在重赋权图上求出的路径就是原图上的最短路径, 无需任何转换? 哪些量需要转换, 哪些量不需要?
3. 若 $\delta'(u, v) = +\infty$, 直接计算 $\text{INF} - h[u] + h[v]$ 会得到什么? 正确写法应当是什么?
4. Johnson 一共调用了几次单源最短路算法? 为什么不能把第 1 步的 Bellman-Ford 也换成 Dijkstra?
5. Johnson 与 Floyd-Warshall 在空间上都需要 $O(n^2)$ 存输出。除输出外, 两者的额外空间分别是多少?
6. 给定 $n = 2000$ 、 $m = 5000$ 且含负权边, 请估算两种算法的基本运算次数并给出选择建议。
7. 若图为无向图且含负权边, Johnson 还适用吗? 一条负权无向边 $\{u, v\}$ 意味着什么? (提示: 把它看作两条有向边。)

4.3 题目要求

实现以下成员函数:

1. `addEdge(u, v, w)`: 加入边表。
2. `allPairs()`: 完整 Johnson 流程, 返回距离矩阵; 存在负权环时抛出 `std::runtime_error`。
3. `path(s, t)`: 在重赋权图上跑一次 Dijkstra 并恢复路径。
4. `pathWeight(path)`: 用原始权值累加路径权值, 用于自检其结果是否等于 `allPairs()[s][t]`。

4.4 接口参考 (仅声明, 补全实现)

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <stdexcept>
5 #include <utility>
6
7 constexpr long long INF = 1LL << 60;
8
9 class Johnson {
10 public:
11     explicit Johnson(int n) : n_(n) {}

```



```

12
13 void addEdge(int u, int v, long long w) {
14     // TODO
15 }
16
17 // 完整 Johnson 流程，返回 n x n 距离矩阵
18 // 步骤提示：
19 // 1. computePotentials() 求 h（负权环则抛 std::runtime_error）；
20 // 2. 用  $w'(u,v) = w + h[u] - h[v]$  建非负权邻接表；
21 // 3. 对每个源点跑 Dijkstra 得到  $dp[u][v] = \delta'(u, v)$ ；
22 // 4. 还原： $dp[u][v] == INF$  时结果为 INF，
23 // 否则  $dist[u][v] = dp[u][v] - h[u] + h[v]$ 。
24 std::vector<std::vector<long long>> allPairs() const {
25     // TODO
26     return {};
27 }
28
29 // 恢复 s -> t 的最短路径；不可达返回 {}
30 std::vector<int> path(int s, int t) const {
31     // TODO
32     return {};
33 }
34
35 // 按原始权值累加给定路径的权值；路径为空返回 INF
36 long long pathWeight(const std::vector<int>& p) const {
37     // TODO
38     return INF;
39 }
40
41 private:
42 std::vector<long long> computePotentials() const {
43     // TODO（可复用第 3 题实现）
44     return {};
45 }
46
47 std::vector<long long> dijkstra(
48     int source,
49     const std::vector<std::vector<std::pair<int, long long>>>& adj,
50     std::vector<int>& parent) const {
51     // TODO（可复用第 2 题实现）

```

```

52     return {};
53 }
54
55 int n_;
56 std::vector<Edge> edges_; // 复用第 3 题的 Edge 结构
57 };

```

4.5 测试用例

```

1 int main() {
2     // 情形 1: 与第 1 题同一张图, 结果必须与 Floyd-Warshall 完全一致
3     Johnson j(5);
4     j.addEdge(0, 1, 3);
5     j.addEdge(0, 2, 8);
6     j.addEdge(0, 4, -4);
7     j.addEdge(1, 3, 1);
8     j.addEdge(1, 4, 7);
9     j.addEdge(2, 1, 4);
10    j.addEdge(3, 0, 2);
11    j.addEdge(3, 2, -5);
12    j.addEdge(4, 3, 6);
13
14    auto d = j.allPairs();
15    for (auto& row : d) {
16        for (long long x : row) std::cout << x << ' ';
17        std::cout << '\n';
18    }
19    // 0  1 -3  2 -4
20    // 3  0 -4  1 -1
21    // 7  4  0  5  3
22    // 2 -1 -5  0 -2
23    // 8  5  1  6  0
24
25    auto p = j.path(0, 2);
26    for (int v : p) std::cout << v << ' ';
27    std::cout << '\n'; // 0 4 3 2
28    std::cout << j.pathWeight(p) << '\n'; // -3, 应等于 d[0][2]
29
30    // 情形 2: 含不可达顶点, 检验 INF 不参与减法

```

```

31     Johnson iso(6);
32     iso.addEdge(0, 1, 3);
33     iso.addEdge(1, 2, -5);
34     // 顶点 3、4、5 孤立
35     auto di = iso.allPairs();
36     std::cout << di[0][2] << '\n';           // -2
37     std::cout << (di[0][3] >= INF) << '\n'; // 1
38     std::cout << (di[3][0] >= INF) << '\n'; // 1
39     std::cout << di[5][5] << '\n';           // 0
40     std::cout << iso.path(0, 4).size() << '\n'; // 0
41
42     // 情形 3: 负权环必须报错
43     Johnson bad(3);
44     bad.addEdge(0, 1, 1);
45     bad.addEdge(1, 2, -1);
46     bad.addEdge(2, 1, -1);
47     try {
48         bad.allPairs();
49     } catch (const std::runtime_error& e) {
50         std::cout << e.what() << '\n'; // 图中存在负权环
51     }
52
53     // 情形 4: 单顶点与带自环
54     Johnson one(1);
55     std::cout << one.allPairs()[0][0] << '\n'; // 0
56
57     Johnson loop(2);
58     loop.addEdge(0, 1, 4);
59     loop.addEdge(1, 1, 3); // 正权自环, 不影响结果
60     std::cout << loop.allPairs()[0][1] << '\n'; // 4
61
62     // 情形 5: 与重复 Dijkstra 交叉验证 (非负权图上两者应完全相同)
63     Johnson jn(4);
64     jn.addEdge(0, 1, 1);
65     jn.addEdge(1, 2, 2);
66     jn.addEdge(2, 3, 3);
67     jn.addEdge(3, 0, 4);
68     jn.addEdge(0, 2, 10);
69     auto dj = jn.allPairs();
70     // 0 1 3 6 / 9 0 2 5 / 7 8 0 3 / 4 5 7 0

```

```
71     std::cout << dj[1][0] << ' ' << dj[2][1] << '\n'; // 9 8
72     return 0;
73 }
```

综合练习：APSP 算法的选择与比较

对下列情形，选择最合适的算法，并说明依据与时间复杂度。

1. $n = 400$ 的稠密图， $m \approx 8 \times 10^4$ ，含负权边，需要完整的距离矩阵。
2. $n = 5000$ 的稀疏图， $m \approx 1.5 \times 10^4$ ，含负权边，需要完整的距离矩阵。
3. 城市路网 $n = 10^5$ 、 $m = 3 \times 10^5$ ，边权为非负实数，只需要 20 个指定站点到所有顶点的距离。
4. 只关心可达性，不关心距离：判断任意两点之间是否存在路径。
5. 需要求出图中权值最小的环（不要求经过指定顶点）。
6. 需要求图的直径，即 $\max_{u,v} \delta(u,v)$ （要求所有点对可达）。
7. 汇率套利检测：判断是否存在一个环使得沿环转换后金额增加。（提示：对汇率取 $-\log$ ，转化为负权环检测。）
8. 图中含负权边但保证无负权环，且需要反复查询任意点对距离。

算法选择表

请补全下表。

图的条件	推荐算法	能否处理负权边	时间复杂度
稠密图，含负权边	_____	_____	_____
稀疏图，边权非负	_____	_____	_____
稀疏图，含负权边	_____	_____	_____
只需可达性（0/1 矩阵）	_____	_____	_____

提高题一：自动选择 APSP 算法

```

1 enum class ApspAlgorithm { FloydWarshall, RepeatedDijkstra, Johnson };
2
3 struct ApspResult {
4     ApspAlgorithm algorithm;
5     std::vector<std::vector<long long>> dist;
6     bool hasNegativeCycle;
7 };
8
9 // 选择规则：

```

```

10 // 1. 若存在负权边且图较稠密 (例如  $m > n * n / 4$ ) -> Floyd-Warshall
11 // 2. 若存在负权边且图较稀疏 -> Johnson
12 // 3. 若所有边权非负且图较稠密 -> Floyd-Warshall
13 // 4. 若所有边权非负且图较稀疏 -> 重复 Dijkstra
14 ApspResult solveApsp(int n, const std::vector<Edge>& edges);

```

请进一步思考：稠密与稀疏的分界阈值应当如何确定？仅靠渐进复杂度是否足够，还需要考虑哪些实际因素（缓存友好性、堆操作常数、`std::vector` 的内存布局）？

提高题二：min-plus 矩阵乘法与重复平方

定义矩阵的 min-plus 乘法

$$(A \otimes B)[i][j] = \min_{0 \leq k < n} (A[i][k] + B[k][j]).$$

设 W 为邻接矩阵（对角线为 0，无边处为 $+\infty$ ），记 $L^{(k)} = W^{\otimes k}$ ，则 $L^{(k)}[i][j]$ 表示最多使用 k 条边时 $i \rightarrow j$ 的最短路径权值。

1. 请说明为什么在无负权环时 $D = L^{(n-1)}$ 。
2. 朴素做法逐次乘出 $L^{(1)}, L^{(2)}, \dots, L^{(n-1)}$ ，复杂度为 $O(n^4)$ 。请用**重复平方**把它降到 $O(n^3 \log n)$ ，并解释为什么 min-plus 乘法满足结合律从而允许快速幂。
3. 实现下列函数，并在第 1 题的图上验证结果与 Floyd-Warshall 一致：

```

1 // min-plus 矩阵乘法，注意跳过 INF 项避免溢出
2 std::vector<std::vector<long long>> minPlus(
3     const std::vector<std::vector<long long>>& A,
4     const std::vector<std::vector<long long>>& B);
5
6 // 用重复平方求 W 的 min-plus 幂，得到 APSP 距离矩阵
7 std::vector<std::vector<long long>> apspByMatrixPower(
8     std::vector<std::vector<long long>> W);

```

4. 为什么这个方案在实践中几乎总是不如 Floyd-Warshall？它在理论上有什么价值（提示：与“最多 k 条边”的受限最短路径、以及次幂方法的可扩展性有关）？

提高题三：Floyd-Warshall 的变形

同一套三重循环稍作修改即可解决一系列问题。请分别写出递推式：

1. 传递闭包： $\text{reach}[i][j]$ 表示可达性，用 `std::bitset` 优化到 $O(n^3/64)$ 。
2. 最小瓶颈路：路径代价定义为路径上最大边权，求使该最大值最小的路径。

3. 最大概率路：边权为 $0 \leq p \leq 1$ 的可靠度，路径代价为各边概率之积，求最大值。
4. 最短路计数：在无负权环且权值为正的图上，统计每对顶点之间最短路径的条数。
5. 最小环：在第 k 轮松弛之前枚举 i, j 并用 $d[i][j] + w(j, k) + w(k, i)$ 更新答案。为什么必须在松弛之前枚举？

附：编写建议与常见误区

1. Floyd-Warshall 的 k 必须在最外层。写成 `for i / for j / for k` 会得到错误结果，因为递推式要求“第 k 轮开始时所有 $d[i][j]$ 都已考虑前 $k-1$ 个中间顶点”。
2. `INF` 不能取 `LLONG_MAX`。`INF + INF` 会溢出为负数，反而“松弛成功”。取 `1LL << 60` 之类的量级，并在相加前判断 `dist[i][k] != INF && dist[k][j] != INF`。
3. 还原 Johnson 距离时先判断不可达。`INF - h[u] + h[v]` 是一个有限的错误值，必须先检查 $\delta'(u, v) = \text{INF}$ 再直接输出 `INF`。
4. Johnson 必须报告任意负权环，而不只是某个源点可达的。虚拟源点能到达所有顶点，因此一次 Bellman-Ford 就够；若换成从某个真实顶点出发，会漏掉其他连通分量里的负权环。
5. 势函数不能用任意真实顶点求。不可达顶点的 h 为 $+\infty$ ，式 (9) 的推导随之失效。
6. `std::priority_queue` 默认是最大堆。必须写成 `priority_queue<State, vector<State>, greater<State>>`，并保持 `State = pair<dist, vertex>` 的字段顺序。
7. 惰性删除不可省略。出堆时 `if (d > dist[u]) continue`；否则过期状态会被重复展开，复杂度退化。
8. 邻接矩阵要处理重边与自环。重边取最小值；权值非负的自环可以忽略，负权自环本身就是负权环。
9. 负权环存在时不要恢复路径。按 `nxt` 跳转会陷入死循环，应先检查 `hasNegativeCycle()` 或限制跳转步数。
10. 对角线初始化为 0，不是 `INF`。否则 $\delta(i, i)$ 会算错，并连带影响所有经过 i 的松弛。
11. 不要用 `int` 存累加距离。 n 条边的权值累加很容易超出 2^{31} ，距离用 `long long`，顶点编号用 `int`。
12. 二维数组按行访问。`vector<vector<long long>>` 的内层连续，三重循环里让最内层遍历 j 可获得更好的缓存局部性；追求性能时可改用一维 `vector` 手工索引。
13. 最短路径不唯一。对照测试时应比较路径权值与距离矩阵是否一致，不要强制要求前驱矩阵或后继矩阵逐元素相同。
14. 渐进复杂度不是唯一标准。Floyd-Warshall 循环体极简、缓存友好，在中等规模稠密图上常常快于理论更优的 Johnson。

建议完成顺序

1. 先写 Floyd-Warshall，重点在三重循环顺序、INF 判断和后继矩阵，把第 1 题情形 1 的距离矩阵作为标准答案。
2. 再写重复 Dijkstra，复用单源最短路专题的实现，在非负权图上与 Floyd-Warshall 交叉验证。
3. 单独完成第 3 题的势函数与重赋权，用断言检查 $w'(u, v) \geq 0$ 对每条边都成立。
4. 把第 2、3 题拼装成完整的 Johnson，在第 1 题的图上验证距离矩阵完全一致。
5. 最后写一个随机图对照测试：随机生成含负权边但无负权环的稀疏图，让 Floyd-Warshall 与 Johnson 输出的矩阵逐元素比较，并统计两者在 $n = 200, 600, 1000$ 时的实际运行时间。